Qwen3-TTS 参考音频预处理调用栈

适用：12Hz-1.7B-Base 的临时参考音频分支；聚焦缓存复用、音频编码和音色提取。

```
prepare_qwen3_tts_base_request()
    │
    ├─ ① get_qwen3_tts_adhoc_reference_service(model, wrapper)
    │    → get_qwen3_tts_adhoc_reference_service_locked()
    │        → 复用已有 ReferenceEncodeService
    │        → 首次使用时创建，并绑定 Qwen3TTSAdhocReferenceHook
    │
    └─ ② reference_service.get_or_encode(state)
        │
        ├─ hook.normalize_input(): 整理参考音频、参考文本、克隆模式
        ├─ hook.cache_key(): 生成缓存键
        │
        ├─ 缓存命中
        │    → hook.load_artifact(): 恢复并返回已有特征
        │
        ├─ 相同参考正在编码
        │    → 等待已有 Future → 复用编码结果
        │
        └─ 需要重新编码
             → ReferenceEncodeService.encode_leader()
             → Qwen3TTSAdhocReferenceHook.encode_one()
                 │
                 ├─ ③ 读取参考音频
                 │    → Qwen3TTSModel._normalize_audio_inputs()
                 │    → Qwen3TTSModel._load_audio_to_np()  ← 字符串输入时
                 │        → librosa.load() / sf.read()
                 │        → 转单声道，得到 waveform、sample_rate
                 │
                 ├─ ④ 提交参考音频编码任务  ← 常规克隆模式
                 │    → Qwen3TTSRefCodeBatcher.submit()
                 │        → 后台线程 run()
                 │        → encode_waveform()
                 │            → 规范化采样率、补齐波形长度
                 │            → CUDA 图重放，或 encoder.encode()
                 │            → ref_code: [音频帧数, 16]
                 │
                 ├─ ⑤ 提取音色向量
                 │    → 按需 librosa.resample()
                 │    → model.extract_speaker_embedding()
                 │    → Qwen3TTSSpeakerEncoderCudaGraphRunner.embed()
                 │        → mel(): 波形 → 对数梅尔频谱
                 │        → CUDA 图重放，或 speaker_encoder.forward()
                 │        → speaker_embedding: [2048]
                 │
                 └─ ⑥ 收集结果
                      → ref_code_future.result(): 等待参考编码完成
                      → 组装 VoicePrompt
                      → 返回 (voice_clone_prompt, ref_text)

get_or_encode() 收到新编码结果（有缓存键时）
    → hook.store_artifact(): 转换成缓存格式
    → hook.revalidate(): 检查是否仍可缓存
    → 可缓存时写入缓存、通知等待者
    → hook.load_artifact(): 恢复结果并返回
    → 返回结果给 prepare_qwen3_tts_base_request()
```

- 无缓存键时，直接编码并返回，不参与缓存和相同请求合并。
- 仅音色模式（x_vector_only_mode=True）跳过④及等待参考编码；ref_code=None。
- ④提交后台任务后继续执行⑤；⑥再等待任务完成，不保证两路 GPU 计算并行。